

Distributed Lexical Analysis via DFA Transition-Function Summaries

The whole project explained simply, then in full detail

Project folder: `pcd/dla` · Course: Principles of Compiler Design

Everything in this guide was checked against the real code, report and result files. Test run today: **62 passed, 0 failed**.

Contents

1. Explained like you are 5
2. The real problem in one picture
3. Background: what is a lexer, a token, a DFA?
4. How the automaton is built (regex → NFA → DFA → small DFA)
5. The big idea: "try every starting state" (3 phases)
6. A worked example, step by step
7. The scan: serial fold vs parallel prefix scan
8. File-by-file tour of the code
9. The two bugs that were found and fixed
10. Test results and benchmark numbers (honest ones)
11. Limitations and future work
12. How to run it yourself · Glossary

1. Explained like you are 5

The story:

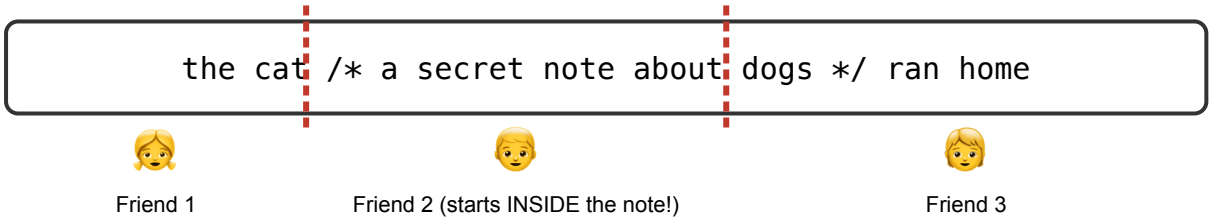
Imagine a super long sentence written on a long paper ribbon. Your job is to circle every **word**. That is slow if one person does it, so you ask 4 friends to help. You cut the ribbon into 4 pieces and each friend circles words in their own piece.

Uh-oh! A scissor cut can land *in the middle of a word*. One friend sees "hel" and the next sees "lo". Worse, imagine a part of the ribbon that says "*Shhh, this is a secret note, ignore everything until the end of the note*". If your piece starts *inside* the secret note, you do not know you are supposed to ignore it, because the beginning of the note is on your friend's piece!

The easy fix would be: first walk the whole ribbon alone to find safe places to cut. But that means one person does a whole walk first. Boring and slow.

This project's clever trick: every friend writes a **cheat-sheet** that says: "If I start my piece while we are in the *normal* mood, I finish in mood A. If I start in the *secret note* mood, I finish in mood B. If I start in the *quote* mood, I finish in mood C..." — one line for every possible mood.

Then a boss lines up the cheat-sheets: "Friend 1 started in normal mood, so he ends in mood A. So Friend 2 starts in mood A. Looking at her sheet, she ends in mood D. So Friend 3 starts in mood D..." No more walking the ribbon! Finally everyone circles their words **for real**, now knowing the right starting mood. The result is exactly the same as if one person did it all.



2. The real problem in one picture

A **lexer** (lexical analyzer) is the first stage of a compiler. It reads raw characters and groups them into **tokens**: `x` (identifier), `=` (assign), `10` (number), `;` (semicolon). For a huge file we want many processes working at once. But a tokenizer is *stateful*: whether the character `/` starts a comment, divides numbers, or is just text inside a string depends on **everything before it**.

Naive splitting (WRONG)

`x = 1; /* long com`	`ment int y = 2; */ z = 3;`

Worker 2 does not know it is inside a comment → reads "ment", "int", "y", "=" … as real code. Extra, bogus tokens!

State-aware splitting (this project: RIGHT)

`x = 1; /* long com`	`ment int y = 2; */ z = 3;`

Worker 2 is told "you start in the INSIDE-COMMENT state" → one comment token, then z = 3 ;

Measured proof: on the 1.5 MB test file, naive splitting produced 18 extra tokens with 2 workers, 88 with 4 workers and 244 with 8 workers. It does not crash, it silently gives the wrong answer, and it gets *worse* with more workers.

3. Background: lexer, token, DFA

Tokens

The toy language ("C-like") has these token kinds (from `lexer_build.py`):

Token	Looks like	Notes
WHITESPACE	spaces, tabs, newlines	recognised then thrown away (skip token)
BLOCK_COMMENT	`/* ... */`	can be thousands of characters long → crosses chunks
LINE_COMMENT	`// ...` to end of line	
STRING	`"text with \" escapes"`	can cross chunks
NUMBER	`42`, `3.14`	
IDENT / KEYWORD	`count`, `if`, `while`	13 keywords (if, else, while, for, return, int, float, string, void, true, false, break, continue); matched as IDENT then relabelled KEYWORD
Operators	`== != <= >= && ||` and `= < > ! + - * /`	2-char ones beat 1-char ones
Punctuation	`( ) { } ; ,`	

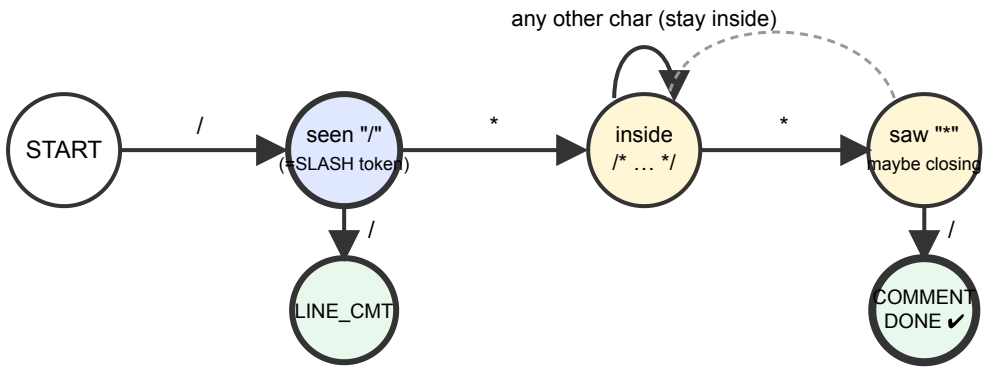
Two rules every lexer follows

Maximal munch = "be greedy". At `>=` the lexer takes the longest possible token, so it is one `>=`, not `>` and `=`. If lengths tie, the lower *priority number* wins (that is why `if` ... is handled via keyword lookup afterwards).

DFA = a board game with one pawn

A DFA (deterministic finite automaton) is like a board game: circles are **moods/states**, arrows are labelled with characters. You put your pawn on START. For each character you read, follow the arrow with that label. Some circles are double-ringed "I just finished a word!" circles. Because every circle has exactly one arrow per character, the future is **fully decided** by (current state + remaining text). That is the property that makes the whole project possible.

Mini picture of the DFA (simplified: just comments and the slash)



The real DFA has **36 states** (42 before minimization) covering all token kinds. "Being inside a block comment" or "inside a string" is literally just *being in particular states*.

4. How the automaton is built

The DFA is **not hand-drawn**. It is generated from regular expressions with the same pipeline that tools like `lex` / `flex` use. Everything is written from scratch in pure Python 3 (no libraries).



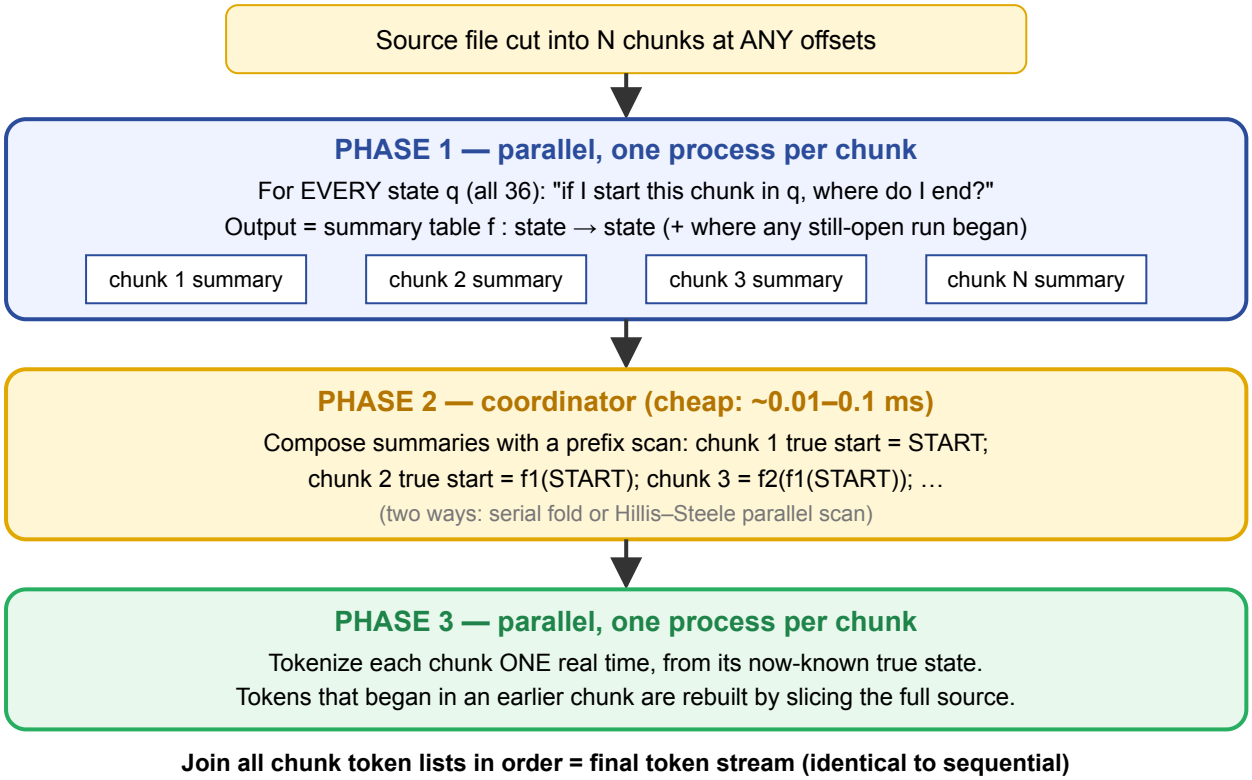
Step by step

1. **Mini regex parser** (`regex_engine.py`): understands literals, escapes (`\n`, `\t`), classes `[a–z]`, `[^"]`, groups `( )`, alternation `|`, and repeats `* + ?`, plus `.` for "any symbol". It turns text into a tree (Lit, Cls, Concat, Alt, Star, Plus, Opt). The alphabet is a fixed set: letters, digits, `_`, whitespace and the punctuation the language uses. Anything else is a lexical error.
2. **Thompson construction** (`automata.py`): each tree node becomes a tiny NFA fragment, glued with free "ε-moves" (moves that consume no character). All token patterns are joined under one start state, and each pattern's final state is labelled with the token name and priority.
3. **Subset construction**: an NFA can be in many places at once. We treat each *set* of NFA states as one DFA state. The empty set is the explicit **dead state** ("no token can match, give up"). If a set contains several accepting NFA states, the one with the lowest priority number names the token.
4. **Minimization** (Moore's partition refinement): states that behave identically for every possible future get merged. Accepting states of different token kinds are kept apart. Result: **42 → 36 states (14.3% smaller)**. A smaller machine means cheaper summaries later (a summary has one row per state).
5. **Terminal states**: states with *no* live outgoing arrow (e.g. right after a lone `;`). Reaching one is always final. This matters for chunk edges (bug #2 below).

> **Example pattern for block comments:** `/\*([^*]|\*+[^*/])*\*+/` — "slash-star, then any mix of non-stars or star-runs that are not followed by a slash, then one-or-more stars and a slash".

5. The big idea: three phases

The code lives in `distributed_lexer.py`. Chunks are cut at **evenly spaced, arbitrary offsets** with no pre-scan. (Workers are processes created with Python's `multiprocessing.Pool`.)



Join all chunk token lists in order = final token stream (identical to sequential)

What exactly is in a summary?

For each chunk, the `ChunkSummary` holds two arrays of length 36:

- `end[q]` — the DFA state you finish in if the chunk is entered in state `q`.
- `open[q]` — if a token/comment/string is still unfinished at the chunk's end, the *absolute file offset* where it began (if that start lies inside this chunk); otherwise nothing. Needed so Phase 3 can rebuild a long comment's full text.

Why composing works (the math, made easy)

> A summary is a function `Q → Q`. Doing chunk A and then chunk B is just function composition: `(B∘A)(q) = B(A(q))`. Composition is **associative** — (A then B) then C equals A then (B then C). Associativity is exactly what a **parallel prefix scan** needs, so the combining can be arranged as a tree instead of a long chain. The code's `combine(a, b)` does this, and also carries the "where did the open run start?" info forward.

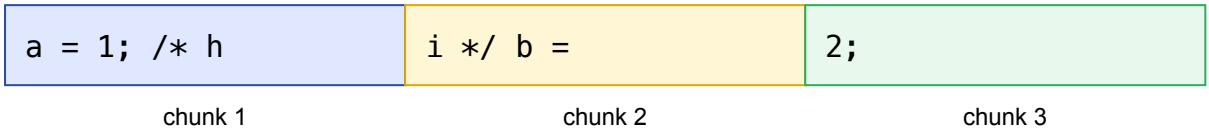
The "closed or open?" rule at a chunk edge

If a chunk ends while the pawn sits on an accepting circle (say after `token123`), the next chunk could start with `4` and extend it. So at a chunk edge, an accepting state is **not final** unless (a) it is

the true end of the file, or (b) it is a *terminal* state that nothing can extend (like `;`). That logic is in `sequential_lexer.tokenize_core`.

6. A worked example

Text (shortened): `a = 1; /* hi */ b = 2;`, cut into 3 pieces:



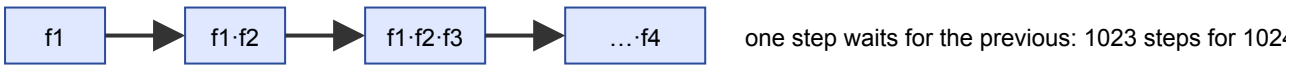
The cut falls INSIDE the comment (after "/* h").

Phase	What happens
1 (parallel)	Chunk 1 assumes START: ends "inside block comment" (call it state C). Chunk 2 computes a row for each of 36 starts. Row "START": it reads `i */ b =` as code → ends at START. Row "inside comment C": the comment closes at `*/`, then `b =` → ends at START. Chunk 3 similar.
2 (coordinator)	Chunk 1 true start = START. Chunk 1 ends in C (and remembers: open run began at offset 7). So chunk 2 true start = **C**, open-start = 7. Chunk 2 under C ends at START, so chunk 3 starts at START.
3 (parallel)	Chunk 2 is told "you begin inside a comment that started at offset 7". It scans `i */`, sees the comment is complete, and builds the lexeme `/* hi */` by slicing the full source from offset 7. Then `b`, `=`. Chunk 3 gives `2`, `;`.

Result: `a = 1 ; BLOCK_COMMENT b = 2 ;` — exactly what a single-threaded lexer produces. The naive method would have wrongly read `i` as an identifier and `*`, `/` as operators.

7. Serial fold vs parallel prefix scan (Phase 2)

Serial fold: depth N−1 (a chain)



Hillis–Steele parallel scan: depth ⌈log₂ N⌉ (doubling)



Each round, element *i* combines with the element *d* places to its left (d = 1, 2, 4…). After ⌈$\log_2 N$⌉ rounds everyone holds the "all chunks up to me" summary. Both methods are implemented (`serial_scan`, `parallel_scan`) and always agree. Measured, with N = 1024 chunks: serial depth 1023 vs 10 parallel rounds — but the parallel version ran **~8× slower in wall-clock time** (12.9 ms vs

1.6 ms) because every round still ran on one process and does O(N log N) total work vs O(N). Fewer steps ≠ faster unless the rounds actually run on separate cores.

8. File-by-file tour

File	Job	Key pieces
`regex_engine.py`	Regex text → tree	ALPHABET, nodes Lit/Cls/Concat/Alt/Star/Plus/Opt, `parse()`, escape table `\n \t \r`
`automata.py`	The "lex" construction	`NFA`, `thompson`, `combine_patterns`, `subset_construction` → `DFA`, `minimize`, `terminal_states`
`lexer_build.py`	The language's token grammar	`TOKEN_SPECS` (26 patterns with priorities), `KEYWORDS`, `build_lexer()` returns raw + minimized DFA, `classify_lexeme`
`sequential_lexer.py`	One-process tokenizer = the **oracle** (ground truth) and the shared core	`Token`, `tokenize_core` (can begin mid-automaton and stop mid-automaton), `tokenize`, `attach_positions` (line/column via binary search)
`distributed_lexer.py`	The 3-phase algorithm	`naive_chunks`, `phase1_build_summaries`, `combine`, `serial_scan`, `parallel_scan`, `true_states_from_scan`, `phase3_tokenize`, `run_distributed`, `run_naive_wrong`
`sourcegen.py`	Makes test source files	Random statements plus 800–6000-char block comments placed to straddle chunk cuts. Sizes: small (200 stmts), medium (2,000), large (12,000)
`tests.py`	Correctness tests	62 checks including a "cut at every single position" test
`benchmark.py`	Runs all experiments	Writes `results/*.csv/json`
`report.md`	The full academic write-up	10 sections + references
`results/`	Saved measurements	`results.csv/json`, `scan_depth.csv`, `naive_wrong.json`, `dfa_sizes.json`
`distributed_lexical_analysis_chat.md`, top `README.md`	Original planning chat; near-empty readme	Background only

Token details

Each `Token` has type, lexeme (text), start/end offsets, and line/column (added at the end). Illegal characters become `ERROR` tokens; an unfinished comment/string at the very end gives `ERROR_UNTERMINATED`.

Why does the sequential lexer double as the building block?

`tokenize_core` takes `start_state` (for resuming), `inherited_open_start` (where the carried-over token began) and `at_eof`. Phase 1 calls it 36× per chunk (throwing tokens away, keeping only end state). Phase 3 calls it once per chunk with the true start state. Using one shared function guarantees both phases agree with the oracle.

9. The two bugs that were found

Bug 1 — "\n" meant the letter n. The regex engine treated a backslash as "take next char literally", so `[^\n]` in the line-comment pattern excluded the letter **n**, not the newline. Every comment containing an "n" got chopped there. Found by testing the raw DFA directly (ruling out minimization). Fix: an escape table for `\n \t \r`.

Bug 2 — a finished token on a chunk edge looked "still open". A `;` landing exactly at the end of a chunk was marked as possibly extendable, so the next chunk was told to "continue" it and flagged its first space as an error. Fix: precompute *terminal states* (nothing can extend them). A sibling bug: a one-letter identifier carried over from the previous chunk was already complete on arrival; the scanner now seeds "best match so far" from the entry state. Regression test: cut the sample at **all 279 positions**, compare to the oracle each time.

Both only appeared on the large benchmark file, a good lesson: tiny hand-written tests are not enough.

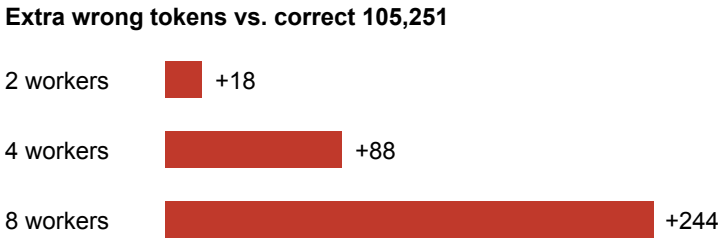
10. Results (real measured numbers)

Correctness

File	Chars	Tokens	Workers	Scan types	Matches oracle?
small	10,818	1,750	1,2,4,8,16	serial, parallel	yes
medium	209,039	17,417	1,2,4,8,16	serial, parallel	yes
large	1,559,709	105,251	1,2,4,8,16	serial, parallel	yes

Everything matches the oracle token-for-token (type, text, start, end). Plus the 62-check test suite (re-run during this explanation: **62 passed, 0 failed**).

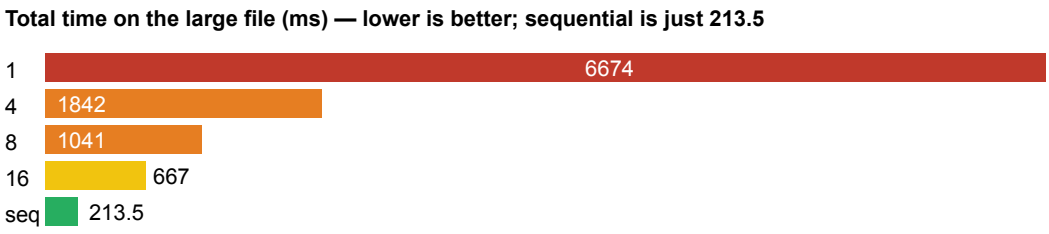
Naive splitting on the large file



Speed (honest result: slower than sequential)

Sequential baseline on the large file: **213.5 ms**.

Workers	Total (ms)	Phase 1 (ms)	Phase 3 (ms)	Speed vs sequential
1	6673.7	6370.6	271.4	0.03×
2	3457.4	3272.1	154.0	0.06×
4	1842.3	1707.9	98.5	0.12×
8	1040.6	926.7	79.7	0.21×
16	667.1	563.8	67.1	0.32×



> **Why slower?** Phase 1 does the work **36 times** (once per state) on each chunk. Even at 16 workers each core still does about 36/16 ≈ 2.25× the entire sequential job's character-scanning, plus the cost of sending text and the DFA to processes (pickling). Phase 1 is ~85% of the time. Speed improves roughly proportionally with more workers but never crosses 1×. The report says so plainly instead of hiding it.

An interesting discovery

On the large file split in 16 chunks, in every chunk **34 of the 36 starting states** end in the same state; only 2 (being inside an unterminated string or inside a block comment) differ. So almost all of the 36× work is redundant, and that points to the main improvement below.

11. Limitations and future work

- **One machine, many processes** — not real networked nodes (the planning chat mentioned FastAPI workers). No network cost is measured.
- **Phase 3 workers receive the whole source** (to slice back-reaching tokens). Simple, but would not scale to files too big for each worker's memory.
- **Small toy language**: no nested comments, no multi-line strings, small fixed operator set; fixed character set (others → errors). Anything regular would work in principle; non-regular features would not.
- **Idea 1 (biggest win):** run from the normal start state only, and do the full 36-way work just for states that fail to converge (only 2 of 36 did) → potentially close to a 10× cut in Phase 1.
- **Idea 2:** dispatch each Hillis–Steele round to real parallel workers so the shorter chain becomes shorter wall-clock time.

12. How to run it · Glossary

```
cd dla
python3 tests.py            # 62 checks, fast, no corpus needed
python3 sourcegen.py corpus # make small/medium/large .src test files
python3 benchmark.py        # full experiments → writes results/
```

Requirements: Python 3 standard library only.

Word	Easy meaning
Lexer / lexical analysis	Chopping program text into meaningful words (tokens)
Token / lexeme	A labelled word / the actual text of it
Regex	A short pattern describing a family of strings
NFA / DFA	Board game where you may (NFA) or may not (DFA) have several choices per move
State	"Mood" of the lexer: e.g. normal, inside comment
Dead state	Trap square: nothing valid can match any more
Maximal munch	Always grab the longest possible token
Chunk	One slice of the file given to one worker
Summary	Worker's cheat-sheet: start state → end state
Prefix scan	Computing "everything up to here" for every position, possibly in parallel
Oracle	The trusted slow sequential lexer used to check answers
Speedup	Sequential time ÷ distributed time (below 1 = slower)

One-sentence summary: workers cut a file anywhere, each prepares a "what if I start in state q?" table, a coordinator chains the tables to learn each chunk's real starting state, and then everyone tokenizes for real — giving exactly the right answer every time, even though in this implementation it is slower than doing the job alone.